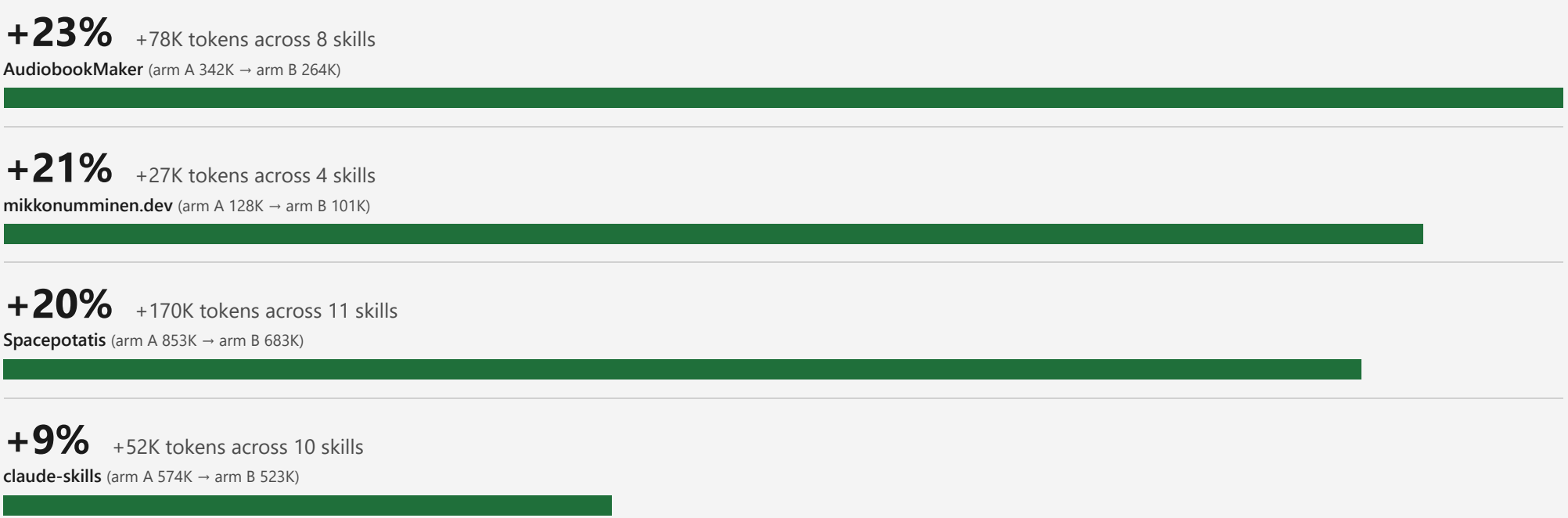
Skill registry — 2026-05-27

A register of every custom skill I've written for Claude Code (Anthropic's coding assistant CLI), and whether it has measured data behind its claims yet. A "skill" here is a reusable slash command — a named recipe Claude Code runs when I type `/audit` or similar. 4 repos, 34 skills, 33 of them A/B-tested for token savings so far.

A/B-MEASURED SAVE RATES BY PORTFOLIO



Aggregate: +17% (+327K tokens across 33 skills). Save rate = (arm A − arm B) / arm A, summed per portfolio. Negative means the skill costs MORE per use than going cold — those skills encode rigor (audit thoroughness, protocol discipline, spec depth), not scout-savings. The 3× heuristic baseline assumed everything would save ~67%; measurement says the truth varies from +52% to −112% depending on what the skill encodes.

Per-skill, four numbers per row: **measured cost / use** (real tokens billed when a skill ran), **measured save / use** (A/B test: arm A minus arm B), **estimated cost / use** (the author's pre-measurement guess), and **estimated save / use** (the 3× heuristic applied to the estimated cost). Green = measured. Italic gray =

estimated. Orange measured-save means the skill cost MORE in the A/B than the unstructured baseline — real findings, not bugs. Cost-or-save dashes (—) mean no data exists for that cell yet.

Per-repo summary

REPO	SKILLS	COST / USE MEASURED	SAVE / USE MEASURED	ESTIMATE-ONLY
AudiobookMaker	8	8	8	0
Spacepotatis	12	11	11	1
claude-skills	10	10	10	0
mikkonumminen.dev	4	4	4	0

All measurement counts are per single invocation of the skill. Cost-measured rows have a real run behind their per-use cost (a transcript-attributed session, or an A/B calibration arm-B). Save-measured rows have a real A/B test behind their per-use savings (calibration). Estimate-only rows have neither yet.

> **SAVE / USE** — tokens saved vs running the same task without the skill, measured by A/B test (arm A cold − arm B with-skill). A **negative value (orange)** means the skill cost more than the unstructured baseline — it encodes rigor, not token compression. **Cost / use** is measured from production transcripts; **save / use** is measured by A/B calibration. They come from different runs — **do not subtract one from the other**.

Claude Code built-ins

reference — not part of the portfolio

Built-in slash commands. Per-use cost is measured from real session transcripts the same way as the custom-skill rows. Per-use save is measured by treating the built-in's prompt as an arm-B recipe in the same A/B methodology used on custom skills — both measured columns are real numbers, not heuristics. The estimated-cost cell stays "—" because built-ins have no SKILL.md author who wrote a guess; the estimated-save cell shows the project-wide 3× baseline heuristic prediction (2× measured cost) so the row carries the same heuristic-vs-measured contrast the custom-skill rows show. Methodology and raw arm-A / arm-B numbers: `docs/audits/builtin-review-calibration-2026-05-22.md` .

SKILL	STATUS	COST / USE *measured*	SAVE / USE *measured*	COST / USE *estimated*	SAVE / USE *estimated*
/review Claude Code built-in PR code review	✓MEASURED last 2026-05-27	**10K** tokens / use *median of 334 runs · mean 48,630 · range 1,152–981K · σ 135K*	**22K** tokens / use (44%) *small-bucket median (typical run) · aggregate 17K (35%) weighted across 11 A/Bs · small 44% · med 26% · large 15% · xlarge -10%*	—	*21K* tokens / use (3× heuristic)

Calibration honesty — where my guesses landed

Here are the rows where I had a guess before I had data. The “How wrong” column is the measurement divided by the guess. Green means I landed within ±10%. Orange means I was off by 5× or more in either direction. 13 of 28 rows are orange. The fix is not to write better guesses next time. The fix is to keep measuring.

Where the guesses come from: each skill carries an author-written estimate in its `SKILL.md` frontmatter (or in the repo's `README.md` table for some skills). I imagined what a single use should cost in tokens and wrote that number down when first scaffolding the skill, before any real run existed. No math, no benchmark — just the number that felt right at the time. The provenance for each row (the `source` tag — either `skill-body` or `readme.md` — and the file path) lives on the row's `prior_estimate` block in `public/data/skills-registry.json`.

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
SPACEPOTATIS **equipment**	4K	157K	**37×** under
AUDIOBOOKMAKER **release-bundle-audit**	3K	87K	**29×** under
AUDIOBOOKMAKER **worktree-launch**	800	23K	**29×** under
SPACEPOTATIS **modular-architecture-audit**	5K	129K	**26×** under
AUDIOBOOKMAKER **ci-failure-triage**	1K	25K	**25×** under
CLAUDE-SKILLS **audit**	425K	28K	**15×** over
SPACEPOTATIS **new-enemy**	6K	68K	**12×** under
AUDIOBOOKMAKER **work-session**	2K	23K	**12×** under
SPACEPOTATIS **new-story**	5K	56K	**10×** under
CLAUDE-SKILLS **ai-codegen-smell-audit**	75K	7K	**10×** over

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
CLAUDE-SKILLS **security-audit**	750K	112K	**6.7×** over
SPACEPOTATIS **new-mission**	8K	50K	**6.2×** under
SPACEPOTATIS **new-perk**	9K	56K	**6.2×** under
AUDIOBOOKMAKER **copyright-scan**	3K	14K	**4.6×** under
SPACEPOTATIS **new-solar-system**	13K	60K	**4.6×** under
AUDIOBOOKMAKER **voice-pack-finnish**	7K	27K	**4.1×** under
SPACEPOTATIS **content-audit**	15K	53K	**3.5×** under
CLAUDE-SKILLS **mikko-install**	4K	12K	**3.1×** under
SPACEPOTATIS **new-migration**	7K	21K	**2.9×** under
CLAUDE-SKILLS **mikko-help**	7K	17K	**2.6×** under
CLAUDE-SKILLS **skill-usage**	4K	10K	**2.5×** under
SPACEPOTATIS **balance-review**	14K	33K	**2.5×** under
CLAUDE-SKILLS **readme-drift-sync**	40K	63K	**1.6×** under
SPACEPOTATIS **save-roundtrip-audit**	12K	19K	**1.5×** under
MIKKONUMMINEN.DEV **md-to-pdf**	25K	21K	**1.2×** over
MIKKONUMMINEN.DEV **sync-readmes**	141K	119K	**1.2×** over

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
MIKKONUMMINEN.DEV **skill-registry**	130K	116K	**1.1×** over
CLAUDE-SKILLS **react-anti-patterns-audit**	23K	22K	**1.0×** within ±10%

Multi-model calibration — does the save rate hold across models?

Same A/B methodology as the per-repo Sonnet calibrations, repeated with sub-agents dispatched as Opus and (later) Haiku. Each skill below was measured at least twice — once per model. The save rate is what changes: Opus is more efficient cold, so the recipe's value compresses; Sonnet rewards scaffolding skills more. Sorted by spread across models — biggest model-sensitivity first.

SKILL	SONNET	OPUS	HAIKU
CLAUDE-SKILLS **readme-drift-sync**	**−112%** arm A 30K → B 63K −33K saved/use	**−28%** arm A 44K → B 56K −12K saved/use	**−6%** arm A 41K → B 44K −3K saved/use
CLAUDE-SKILLS **session-cost**	**+14%** arm A 36K → B 31K 5K saved/use	**+4%** arm A 33K → B 32K 1K saved/use	**−89%** arm A 30K → B 56K −26K saved/use
AUDIOBOOKMAKER **copyright-scan**	**+49%** arm A 65K → B 33K 32K saved/use	**−20%** arm A 46K → B 55K −9K saved/use	**+57%** arm A 61K → B 27K 35K saved/use
CLAUDE-SKILLS **skill-usage**	**+53%** arm A 49K → B 23K 26K saved/use	**−16%** arm A 36K → B 42K −6K saved/use	**+57%** arm A 64K → B 28K 36K saved/use
SPACEPOTATIS **new-migration**	**+16%** arm A 25K → B 21K 4K saved/use	**+36%** arm A 52K → B 33K 19K saved/use	**−33%** arm A 33K → B 44K −11K saved/use
SPACEPOTATIS **new-solar-system**	**+9%** arm A 65K → B 60K 6K saved/use	**+73%** arm A 128K → B 35K 93K saved/use	**+16%** arm A 48K → B 41K 8K saved/use
SPACEPOTATIS **new-enemy**	**+9%** arm A 75K → B 68K 7K saved/use	**+70%** arm A 117K → B 35K 82K saved/use	**+57%** arm A 62K → B 27K 35K saved/use
SPACEPOTATIS **equipment**	**−5%** arm A 89K → B 94K −5K saved/use	**+48%** arm A 85K → B 44K 41K saved/use	**−3%** arm A 44K → B 45K −1K saved/use

SKILL	SONNET	OPUS	HAIKU
SPACEPOTATIS **content-audit**	**+22%** arm A 99K → B 77K 22K saved/use	**−25%** arm A 79K → B 99K −20K saved/use	**−2%** arm A 60K → B 61K −912 saved/use
CLAUDE-SKILLS **ai-codegen-smell-audit**	**+48%** arm A 157K → B 81K 76K saved/use	**+10%** arm A 112K → B 101K 11K saved/use	**+1%** arm A 61K → B 60K 496 saved/use
CLAUDE-SKILLS **mikko-help**	**−22%** arm A 24K → B 29K −5K saved/use	**−21%** arm A 33K → B 40K −7K saved/use	**−62%** arm A 20K → B 33K −12K saved/use
SPACEPOTATIS **balance-review**	**+28%** arm A 46K → B 33K 13K saved/use	**−12%** arm A 32K → B 36K −4K saved/use	**+5%** arm A 38K → B 36K 2K saved/use
AUDIOBOOKMAKER **ci-failure-triage**	**+43%** arm A 44K → B 25K 19K saved/use	**+10%** arm A 31K → B 28K 3K saved/use	**+14%** arm A 44K → B 38K 6K saved/use
SPACEPOTATIS **modular-architecture-audit**	**+26%** arm A 175K → B 129K 46K saved/use	**−4%** arm A 100K → B 104K −4K saved/use	**+29%** arm A 46K → B 33K 13K saved/use
CLAUDE-SKILLS **mikko-install**	**−35%** arm A 17K → B 23K −6K saved/use	**−2%** arm A 27K → B 27K −503 saved/use	**−14%** arm A 23K → B 26K −3K saved/use
AUDIOBOOKMAKER **pronunciation-corpus-add**	**+12%** arm A 19K → B 17K 2K saved/use	**+39%** arm A 42K → B 25K 17K saved/use	**+7%** arm A 24K → B 22K 2K saved/use
AUDIOBOOKMAKER **release-bundle-audit**	**−12%** arm A 78K → B 87K −9K saved/use	**−1%** arm A 80K → B 80K −546 saved/use	**+20%** arm A 52K → B 41K 11K saved/use
AUDIOBOOKMAKER **voice-pack-finnish**	**+52%** arm A 56K → B 27K 29K saved/use	**+20%** arm A 45K → B 36K 9K saved/use	**+32%** arm A 48K → B 33K 15K saved/use

SKILL	SONNET	OPUS	HAIKU
MIKKONUMMINEN.DEV **md-to-pdf**	**+41%** arm A 35K → B 21K 15K saved/use	**+22%** arm A 36K → B 28K 8K saved/use	**+54%** arm A 59K → B 27K 32K saved/use
AUDIOBOOKMAKER **release-cut**	**+17%** arm A 35K → B 29K 6K saved/use	**+48%** arm A 65K → B 34K 31K saved/use	**+29%** arm A 55K → B 39K 16K saved/use
CLAUDE-SKILLS **audit**	**+2%** arm A 117K → B 114K 3K saved/use	**−7%** arm A 216K → B 231K −15K saved/use	**+23%** arm A 92K → B 71K 21K saved/use
CLAUDE-SKILLS **mikko-audit-suite**	**−21%** arm A 20K → B 24K −4K saved/use	**+5%** arm A 35K → B 33K 2K saved/use	**+8%** arm A 37K → B 34K 3K saved/use
AUDIOBOOKMAKER **worktree-launch**	**+1%** arm A 23K → B 23K 126 saved/use	**+24%** arm A 34K → B 26K 8K saved/use	**+23%** arm A 38K → B 30K 9K saved/use
MIKKONUMMINEN.DEV **sync-readmes**	**−11%** arm A 33K → B 37K −4K saved/use	**−1%** arm A 58K → B 59K −680 saved/use	**−24%** arm A 46K → B 56K −11K saved/use
SPACEPOTATIS **new-mission**	**+37%** arm A 79K → B 50K 29K saved/use	**+15%** arm A 53K → B 45K 8K saved/use	**+19%** arm A 41K → B 33K 8K saved/use
MIKKONUMMINEN.DEV **skill-localUpdate**	**+52%** arm A 44K → B 21K 23K saved/use	**+39%** arm A 50K → B 31K 19K saved/use	**+33%** arm A 46K → B 31K 15K saved/use
CLAUDE CODE BUILT-IN **/review**	**+44%** arm A 50K → B 28K 22K saved/use	**+34%** arm A 53K → B 35K 18K saved/use	**+52%** arm A 58K → B 28K 30K saved/use
CLAUDE-SKILLS **react-anti-patterns-audit**	**−33%** arm A 17K → B 22K −5K saved/use	**−28%** arm A 24K → B 31K −7K saved/use	**−21%** arm A 21K → B 25K −4K saved/use

SKILL	SONNET	OPUS	HAIKU
CLAUDE-SKILLS **security-audit**	**−4%** arm A 108K → B 112K −4K saved/use	**−8%** arm A 120K → B 130K −10K saved/use	**+3%** arm A 72K → B 69K 2K saved/use
SPACEPOTATIS **new-perk**	**+32%** arm A 82K → B 56K 27K saved/use	**+25%** arm A 46K → B 34K 12K saved/use	**+30%** arm A 54K → B 38K 16K saved/use
MIKKONUMMINEN.DEV **skill-registry**	**−38%** arm A 16K → B 22K −6K saved/use	**−39%** arm A 24K → B 33K −9K saved/use	**−32%** arm A 20K → B 27K −6K saved/use
SPACEPOTATIS **new-story**	**+29%** arm A 47K → B 33K 14K saved/use	**+34%** arm A 84K → B 55K 29K saved/use	**+28%** arm A 60K → B 43K 16K saved/use
SPACEPOTATIS **save-roundtrip-audit**	**+10%** arm A 70K → B 63K 7K saved/use	**+15%** arm A 91K → B 78K 13K saved/use	**+16%** arm A 70K → B 59K 11K saved/use
AUDIOBOOKMAKER **work-session**	**−10%** arm A 21K → B 23K −2K saved/use	**−10%** arm A 30K → B 33K −3K saved/use	**−12%** arm A 24K → B 27K −3K saved/use

Each cell shows the A/B save rate (% of arm A tokens), the raw arm-A → arm-B tokens, and the absolute saved-per-use. Orange = the skill cost MORE on this model than the unstructured baseline. Cells marked † had a procedure deviation in arm B — see the footnote block below. Sample size N=1 per (skill, model) pair — trust direction and magnitude, not two-significant-digit precision. The cross-portfolio observation is that recipe value shrinks as model capability rises: skills that save 50%+ on Sonnet typically settle at 20-40% on Opus because the cold arm gets cheaper, not because the recipe arm gets more expensive.

Per-repo skill tables

One row per skill. Measured rows have a green wash; estimated rows are white. Four data columns per row, in pairs: **Cost / use** measured + estimated, then **Save / use** measured + estimated. Green numbers come from real runs (transcript or A/B). Italic gray numbers are author estimates. Orange numbers are real A/B measurements showing the skill cost MORE per use than the unstructured baseline. Dashes mean no data exists for that cell yet. Every figure is per single invocation — no annual projections anywhere in this document.

> **SAVE / USE** — tokens saved vs running the same task without the skill, measured by A/B test (arm A cold − arm B with-skill). A **negative value (orange)** means the skill cost more than the unstructured baseline — it encodes rigor, not token compression. **Cost / use** is measured from production transcripts; **save / use** is measured by A/B calibration. They come from different runs — **do not subtract one from the other**.

AudiobookMaker

8 skills · 8 measured · github.com/MikkoNumminen/AudiobookMaker

SKILL	STATUS	COST / USE *measured*	SAVE / USE *measured*	COST / USE *estimated*	SAVE / USE *estimated*
ci-failure-triage When CI fails on a release tag or PR, walk the failure modes in the right order against a recipe library built from the project's fix(ci) commit history.	✓MEASURED last 2026-05-23	**25K** tokens / use	**19K** tokens / use	*1K* tokens / use	*2K* tokens / use
copyright-scan Scan a git diff (staged by default, or a named revision range / file set) for accidental third-party copyright leaks before they land on origin.	✓MEASURED last 2026-05-12	**14K** tokens / use	**32K** tokens / use	*3K* tokens / use	*6K* tokens / use
pronunciation-corpus-add Append a Finnish pronunciation failure (a word the Chatterbox Grandmom voice mispronounces) to the project's pronunciation corpus at docs…	✓MEASURED last 2026-05-23	**17K** tokens / use	**2K** tokens / use	—	—
release-bundle-audit Audit the AudiobookMaker PyInstaller release bundle for unused dependencies, dead-code data files, and accidental ML-stack pollution; then propose spec-only fixes on a `chore/release-bundle-size` branch.	✓MEASURED last 2026-05-23	**87K** tokens / use	**−9K** tokens / use	*3K* tokens / use	*6K* tokens / use
release-cut Cut a new AudiobookMaker release (bump APP_VERSION, tag vX.Y.Z, push, verify CI lands SHA-256 in release notes AND sidecar). Use this ski…	✓MEASURED last 2026-05-12	**32K** tokens / use *median of 6 runs · mean 31,377 · range 18,416–48,979 · σ 9,595*	**6K** tokens / use	—	—

voice-pack-finnish Build a Finnish voice pack from a source audio file end-to-end — chunked analyze with ECAPA diarization, transcript validation per speaker, branching to LoRA training (long sources) or few-shot ref-clip packaging (short sources), and ear-check by synth.	✓MEASURED last 2026-05-23	**27K** tokens / use	**29K** tokens / use	*7K* tokens / use	*13K* tokens / use
work-session Start, pause, or finish a TODO.md work session as one of the 4 permanent Claude sessions in AudiobookMaker. Use this whenever the user sa…	✓MEASURED last 2026-05-23	**23K** tokens / use	**−2K** tokens / use	*2K* tokens / use	*4K* tokens / use
worktree-launch Start a new parallel Claude session safely.	✓MEASURED last 2026-05-23	**23K** tokens / use	**126** tokens / use	*800* tokens / use	*2K* tokens / use

Spacepotatis

12 skills · 11 measured · github.com/MikkoNumminen/Spacepotatis

SKILL	STATUS	COST / USE *measured*	SAVE / USE *measured*	COST / USE *estimated*	SAVE / USE *estimated*
balance-review Diff uncommitted changes to game data (weapons, enemies, waves, missions, perks, augments, loot pools, solar systems) and report DPS, TTK, energy-cost-per-DPS, augment-folded effective DPS, loot-pool roster shifts, and drop-rate deltas vs HEAD.	✓MEASURED last 2026-05-22	**33K** tokens / use	**13K** tokens / use	*14K* tokens / use	*27K* tokens / use
content-audit Pre-commit content invariants check — orphan refs (enemy / weapon / sprite / pod / loot-pool / mission system / story trigger), missing sprite generators, perk drop-weight sanity, mission prereq DAG, story integrity (voice + music files, trigger refs), storyTriggers helper coverage.	✓MEASURED last 2026-05-03	**53K** tokens / use	**22K** tokens / use	*15K* tokens / use	*30K* tokens / use
equipment Create, modify, or remove a weapon or piece of equipment (augment / reactor / shield / armor) — including visual changes to how it appears in combat or in the loadout UI.	✓MEASURED last 2026-05-03	**157K** tokens / use *median of 3 runs · mean 268K · range 26,556–619K · σ 254K*	**−5K** tokens / use	*4K* tokens / use	*9K* tokens / use
modular-architecture-audit Orchestrates the multi-phase modular-architecture audit + refactor.	✓MEASURED last 2026-05-22	**129K** tokens / use	**46K** tokens / use	*5K* tokens / use	*10K* tokens / use
new-enemy Scaffold a new enemy — enemies.json entry, BootScene placeholder sprite generator, optional test wave, and integrity test verification.	✓MEASURED last 2026-05-22	**68K** tokens / use	**7K** tokens / use	*6K* tokens / use	*11K* tokens / use

new-migration Add a Postgres schema migration end-to-end — dated SQL file in db/migrations/, Database interface update in src/lib/db.ts, prod applicati…	✓MEASURED last 2026-05-22	**21K** tokens / use	**4K** tokens / use	*7K* tokens / use	*14K* tokens / use
new-mission Scaffold a new combat mission across missions.json, waves.json, the galaxy planet binding, and a smoke test — in one shot.	✓MEASURED last 2026-05-22	**50K** tokens / use	**29K** tokens / use	*8K* tokens / use	*16K* tokens / use
new-perk Scaffold a new mission-only perk — perks.ts entry, BootScene icon generator, HUD chip wiring, and (for actives) a switch case in PerkCont…	✓MEASURED last 2026-05-22	**56K** tokens / use	**27K** tokens / use	*9K* tokens / use	*18K* tokens / use
new-solar-system Add a new solar system to the galaxy — solarSystems.json entry (incl. galaxy bed), SolarSystemId union extension, optional unlock gating,…	✓MEASURED last 2026-05-22	**60K** tokens / use	**6K** tokens / use	*13K* tokens / use	*26K* tokens / use
new-story Add, modify, or remove story content — cinematic popups, voiceovers, music beds, body text, written synopses, and auto-trigger wiring.	✓MEASURED last 2026-04-30	**56K** tokens / use	**14K** tokens / use	*5K* tokens / use	*11K* tokens / use
new-weapon *Superseded by /equipment.*	REDIRECT	—	—	—	—
save-roundtrip-audit Pre-commit save-pipeline invariants check — walks every StateSnapshot field through the 8 layers of the save round-trip (snapshot interfa…	✓MEASURED last 2026-05-03	**19K** tokens / use	**7K** tokens / use	*12K* tokens / use	*24K* tokens / use

claude-skills

10 skills · 10 measured · github.com/MikkoNumminen/claude-skills

SKILL	STATUS	COST / USE *measured*	SAVE / USE *measured*	COST / USE *estimated*	SAVE / USE *estimated*
ai-codegen-smell-audit Read-only audit for specific failure modes that recur in LLM-generated code — defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on.	✓MEASURED last 2026-05-17	**7K** tokens / use *median of 4 runs · mean 45,444 · range 2,407–165K · σ 68,864*	**76K** tokens / use	*75K* tokens / use	*150K* tokens / use
audit Run a multi-phase robustness audit of the current codebase.	✓MEASURED last 2026-05-20	**28K** tokens / use *median of 6 runs · mean 153K · range 4,051–670K · σ 239K*	**3K** tokens / use	*425K* tokens / use	*850K* tokens / use

mikko-audit-suite Run every `mikko-*` audit skill that's relevant to the current codebase, in a sensible order, and produce one index document linking to each audit's report.	✓MEASURED last 2026-05-23	**24K** tokens / use	**−4K** tokens / use	—	—
mikko-help List every installed `mikko-*` skill with its description (or with its `barney:` field when `--barney` is passed).	✓MEASURED last 2026-05-20	**17K** tokens / use *median of 9 runs · mean 143K · range 8,271–528K · σ 185K*	**−5K** tokens / use	*7K* tokens / use	*13K* tokens / use
mikko-install Install, update, list, or uninstall `mikko-*` skills from the cloned `claude-skills` source repo into the user-wide skill directory (`~/....	✓MEASURED last 2026-05-21	**12K** tokens / use *median of 3 runs · mean 15,824 · range 12,169–22,821 · σ 4,950*	**−6K** tokens / use	*4K* tokens / use	*8K* tokens / use
react-anti-patterns-audit Read-only audit for six concrete React anti-patterns that recur in component code (key=index in lists, useEffect for derived state, dep-array lies, state mutation instead of replace, missing effect cleanup, multiple sources of truth).	✓MEASURED last 2026-05-23	**22K** tokens / use	**−5K** tokens / use	*23K* tokens / use	*45K* tokens / use
readme-drift-sync Detect drift between what the repo actually contains and what `README.md` claims, then rewrite ONLY the drifted sections in the README's ...	✓MEASURED last 2026-05-23	**63K** tokens / use	**−33K** tokens / use	*40K* tokens / use	*80K* tokens / use
security-audit Orchestrates the multi-phase security audit + remediation.	✓MEASURED last 2026-05-22	**112K** tokens / use	**−4K** tokens / use	*750K* tokens / use	*1.50M* tokens / use
session-cost Measure the token cost of a single Claude Code session — main thread plus every sub-agent it dispatched.	✓MEASURED last 2026-05-21	**409K** tokens / use	**5K** tokens / use *vs 35,880 A/B baseline · 14%*	*0* tokens / use	*0* tokens / use
skill-usage Measure actual Claude Code skill usage from local transcript JSONL files.	✓MEASURED last 2026-05-21	**10K** tokens / use *median of 5 runs · mean 14,926 · range 5,781–38,730 · σ 12,067*	**26K** tokens / use	*4K* tokens / use	*8K* tokens / use

mikkonumminen.dev

4 skills · 4 measured · github.com/MikkoNumminen/mikkonumminen.dev

SKILL	STATUS	COST / USE *measured*	SAVE / USE *measured*	COST / USE *estimated*	SAVE / USE *estimated*
md-to-pdf Render a markdown report (or any HTML) to a styled PDF using the local Chrome's `--print-to-pdf` flag.	✓MEASURED last 2026-05-23	**21K** tokens / use	**15K** tokens / use	*25K* tokens / use	*50K* tokens / use

skill-localUpdate Refresh every local artifact the site renders about your portfolio skills — the JSON the /contact terminal fetches at runtime, the measurement-overlaid annual totals, and the downloadable skills-registry PDF — in one sequenced chain.	✓MEASURED last 2026-05-23	**21K** tokens / use	**23K** tokens / use	—	—
skill-registry Scan every sibling repo under D:/koodaamista for `.claude/skills/*/SKILL.md` files and emit a consolidated registry as a structured JSON …	✓MEASURED last 2026-05-20	**116K** tokens / use *median of 4 runs · mean 111K · range 82,778–128K · σ 16,836*	**−6K** tokens / use *vs 16,250 A/B baseline · -38%*	*130K* tokens / use	*260K* tokens / use
sync-readmes Audit project data against sibling repos' READMEs and open a PR with drift corrections.	✓MEASURED last 2026-05-18	**119K** tokens / use	**−4K** tokens / use *vs 32,885 A/B baseline · -11%*	*141K* tokens / use	*282K* tokens / use

Method, in plain language

How the numbers in this document are produced, what they mean, and where I am still guessing.

Per single use — the only unit on this page

Every number in the document is per one invocation of the skill. There are no annual figures, no monthly figures, no "tokens per year" anywhere. The original document had those; they were models stacked on guesses about how often a skill gets used. This version drops them. If you want to know what a skill costs you over a year, multiply the per-use number by however many times you'll actually run it — the model wasn't going to be more honest than that anyway.

How “measured cost / use” is produced

Two sources, both real. **Transcript measurement**: every Claude Code session writes a JSON-Lines transcript to `~/.claude/projects/<dir>/<sessionId>.jsonl`, with each assistant message carrying an `attributionSkill` field when a skill is active. The `skill-usage` skill walks those files, groups by skill, sums `input_tokens + output_tokens + cache_creation_input_tokens` (cache reads excluded — those are paid upstream), dedupes by `requestId`, and reports per-use averages across whatever invocations landed in the 90-day window. That's "what happened in production."

A/B calibration arm-B: when a skill hasn't been invoked in production (no transcript data), an A/B run still spends real tokens — a Sonnet sub-agent followed the `SKILL.md` end-to-end on a representative task, with usage billed through the harness. Arm-B IS a real cost-per-use measurement, just from a controlled run instead of in-the-wild use. Rows in this state show the cost with a "tokens / use (A/B-measured)" sub-label. Rows with both transcript and A/B data prefer the transcript number — it's what happened, not what's reproducible.

What counts as one “use” — the invocation-boundary correction

The upstream `skill-usage` parser groups assistant messages by `(skill, sessionId)`. Every assistant message attributed to one skill inside one Claude Code session counts as part of *one* invocation. That's accurate for total tokens spent and last-invoked timestamps — but it *overstates* tokens-per-use whenever a single session contains multiple uses of the same skill. For `/review` that's a 23× distortion: 14 sessions contained 336 distinct `/review` calls, so the parser's session-grouped "avg 1.15M / use" was really "avg cost of 336 uses spread across 14 sessions, divided by 14 instead of by 336."

This document corrects for that. `build-review-stats.mjs` walks the `parentUuid` → originating-user-message chain on every transcript-measured row and uses each user message's `promptId` as the invocation ID. Each distinct `(sessionId, promptId)` is one use. For `/review` that drops the per-use figure from 1.15M to ~10K

(median); the 90-day total spend is unchanged, only its decomposition into *per use* × *uses*. For other heavily-iterated skills the correction is much smaller (most have one or two sessions in the window, so session-grouped ≈ invocation-grouped), but every row with N≥2 invocations gets the same accounting so the comparison stays apples-to-apples.

Median, not mean, on the cost-per-use headline

Wherever a row's cost-per-use is the average of multiple invocations, the headline number is the **median** — the cost distribution is heavily right-skewed (one or two large invocations pull the mean well above what one typical use costs), so the median is the honest "what does one use of this skill cost" figure. The mean, range, and σ ride in the sub-cell as honest spread information; if the σ is comparable to the median, treat the headline as a soft anchor and look at the spread to understand the variance. Single-invocation rows (A/B-only or one transcript hit) don't have a distribution, so they keep their unadorned headline.

How “measured save / use” is produced

One source: a calibration A/B test. Two Sonnet sub-agents solve the same task in fresh sandboxed worktrees — arm A cold (no `SKILL.md` access), arm B following the skill. Save / use is arm-A tokens minus arm-B tokens for that one run. **N = 1 per skill, single data point**. A re-run would produce different absolute numbers for both arms; trust direction and rough magnitude, not two-significant-digit precision.

Some skills show negative save / use in orange. Those are real findings: the skill arm spent MORE tokens than the unstructured arm, because the skill encodes rigor (e.g. a full-CRUD lifecycle or a multi-phase audit) that the unstructured arm skipped. The skill's value is completeness, not token compression. The arm-A / arm-B numbers are preserved on each calibrated row's receipt for any downstream consumer that wants to see both sides.

Exception: `/review`. N=11 A/Bs, bucketed by PR size. The original N=1 measurement on a 5-file PR (63% saved) was the upper end of a sharp size gradient: re-running on real production PRs of 174–3977 lines reveals the recipe saves most on small PRs and actively *costs more* on the largest ones. Bucket medians: small (0–199 lines) **44%**, medium (200–799) **26%**, large (800–2499) **15%**, extra-large (2500+) **−10%**. 63% of production /review invocations are on small PRs, so the median production run lives in that bucket — the row's headline save (22K, 44%) is the small-bucket median, picked to describe the same typical run as the headline cost. The across-bucket aggregate (17K, 35%, weighted by production frequency) sits in the row's sub-label for readers who want the population-level figure. The per-bucket and per-PR data live on the row's `calibration_ab_buckets` + `calibration_ab_runs` arrays for downstream consumers. Other skills stay at N=1 until they accumulate enough production usage to warrant a multi-PR pass.

Different regimes. Cost on the `/review` row is from production transcripts (real invocations summarised over the 90-day window). Save is from A/B calibrations (synthetic cold-vs-recipe pairs on representative PRs). They

measure related but distinct things: cost is what one production use spends; save is what one matched A/B pair would save. A reader must not divide save by cost to get a "%" — the % shown is anchored to the A/B baseline, not the production cost. Same caveat applies to every transcript-measured row in the document; the row-level subline ("vs <armA> A/B baseline · pct") makes the anchoring explicit on rows where the regime gap is large.

Regime gap: when measured cost and measured save come from different scales

Several rows pair a transcript-measured cost (the average of N real production invocations) with an A/B-measured save (a single calibration run on a deliberately-small representative task). When the production runs are *much larger* than the A/B task — and they often are, by 5–15× — the cost and save sit in different regimes. Reading the row as "save / cost = recipe efficiency" gives the wrong answer.

Concrete: an early version of this document showed `/review` as **1.15M tokens / use measured cost** next to **~24K tokens / use measured save**. The cost was a session-grouped artifact (14 sessions, 336 actual invocations) and the save was a single small-PR A/B (~60K baseline). Doing 24K ÷ 1.15M would read as a 2% save rate; the actual finding from the A/B was ~39%. Both numbers were real, but they sat in different regimes — the cost was production-scale, the save was calibration-scale. The current `/review` row pulls both numbers to the same point of the distribution: cost is the per-invocation production median (~10K), save is the small-bucket A/B median (~22K, 44% off a ~50K arm-A). The save still sits at the calibration scale rather than the production scale — production /review tasks run smaller than even the smallest A/B PR — so the absolute save figure overstates the tokens you would have spent, but the 44% rate describes the typical small-bucket A/B run truthfully, and the row's headline can be parsed end-to-end without mixing distribution points. The same trap still shows up on roughly a dozen other transcript-measured rows where the production-scale cost runs 2× or more above its A/B baseline — common offenders include `mikko-help`, `session-cost`, `equipment`, `audit`, `release-cut`, and `skill-registry`. Every row that hits the threshold gets the same labelling treatment described next.

When a row hits this regime gap (transcript cost > 2× the A/B arm-A baseline) the measured-save cell prints a subline making the baseline explicit: *vs <armA> A/B baseline · <pct>%*. Math on the row now works: `save ÷ baseline = pct` instead of `save ÷ visible-cost = misleading`. This isn't a correction — both numbers were always real. It's a labelling fix so a reader doesn't combine them wrong.

The gap itself is the finding: **the A/B calibration task isn't representative of what production runs of these skills actually look like**. The honest read is that recipe value scales with task complexity, and the A/B numbers underestimate the absolute save at production scale (the same recipe collapsing the same amount of structure, applied to a 1M-token task instead of a 70K-token task, would save proportionally more). The fix is either re-running calibrations on representative-sized targets, or treating the A/B save as a lower bound. I haven't done the former; the document treats the A/B save as what it is.

How “estimated cost / use” is produced

> I made up the number. I imagined what running the skill should cost in tokens and wrote that down. There's no math behind these. They are guesses by the person who built the skill, written down before any measurement existed.

The estimates stay in the document so the picture covers every skill, not just the ones I've measured. Every measured estimate I had has turned out to be off, usually low by 5× to 100× — apply the same skepticism to the rows that haven't been measured yet. If anything, the un-measured estimates are likely to be *more* wrong: I measured the ones I felt most confident about first.

How “estimated save / use” is produced

3× heuristic on the estimated cost: *saved ≈ 2× cost*. The model says "doing this without the skill would cost about 3× what the skill costs," so the saved-per-use is 2× the cost-per-use. The 3× is a handful-of-side-by-side-runs guess; the May-2026 Spacepotatis calibration showed it's overstated by roughly 3× at the portfolio level (measured savings averaged ~22% of arm-A cost, vs the heuristic's ~67%). Treat estimated savings as a possibly-too-optimistic upper bound.

Cost is on both sides of the underlying subtraction, so the estimated save figure is more sensitive to bad cost estimates than the estimated cost is. An italic estimated-save stacked on top of an italic estimated-cost is a model on top of a guess — least trustworthy cell on the page. A green measured-save on top of a green measured-cost is the most trustworthy. The visual treatment matches that hierarchy.

What this document does NOT claim

- No production cost numbers. This is development-time tooling on my own machine.
- No comparison to other engineers' setups. I only have transcripts for one engineer.
- No per-feature ROI claim. This is per-skill cost, not per-feature value.
- No assertion that any of this scales linearly to a team. It might. I have not measured.
- No claim that the modeled savings would survive an actual A/B test against well-prompted unstructured chats. They might; I haven't tested.

Reproducibility

From inside this repo (`mikkonumminen.dev/`) on a machine with Claude Code installed:

1. `/mikko-skill-usage` — writes `.claude/agent-verdicts/SKILL-USAGE-LATEST.json` from local transcripts.

2. `/skill-localUpdate` (formerly `/skill-pdf`) — re-walks the portfolio inventory, applies the measurement overlay, renders this PDF.

The data the renderer reads is committed to the repo at `public/data/skills-registry.json`. If you want to inspect what's behind a number on this page, that file is the source of truth. The dated history lives in `.claude/agent-verdicts/SKILL-REGISTRY-*.json`.

The *last* <*date*> marker on a measured row is the timestamp of the most recent Claude Code transcript that invoked that skill. The transcript itself stays on my machine because it contains code and context from private repos. If you want to verify the measurement methodology rather than the measurement, the parser source is at `~/.claude/skills/mikko-skill-usage/SKILL.md` — it is a JSONL parser, not a network call to anything.
